

CSE420

Compiler Design

Topic

Type Grammar and Attributes of a Type

and

Type Information Processing During Bottom-up Parsing

Prepared by Azmari Sultana

Table of Contents

Type Grammar and Attributes of a Type

1. Type Expressions.....	2
2. Type Equivalence.....	3
3. A Grammar for Types and Declarations.....	4
4. Attributes for the Type Grammar.....	5

Type Information Processing During Bottom-up Parsing

5. From Inherited Attributes to Ordinary Variables.....	8
6. Width of a Type and Storage Layout.....	10
7. Sequences of Declarations.....	11
8. Fields in Records and Classes.....	13
9. Single-Pass Bottom-Up Processing — Practical Implications.....	28

Practice Problems Included in This Note

Final Examination, Spring 2025, Set A — Question 1 (Symbol-Table Attributes).....	12
Final Examination, Fall 2025, Set A — Question 8 (Records, Alignment).....	14
Final Exam, Summer 2025, Set A — Question 1.1 (Nested Records).....	18
Final Exam, Spring 2026, Set A — Question 7 (Record Symbol Table).....	22
Final Exam, Spring 2026, Set A — Question 4 (Struct Offsets, TAC, Inheritance).....	25
Final Examination, Fall 2025, Set A — Question 5 (Single-Pass Processing).....	29

1. Type Expressions

A compiler needs a precise way to represent the type of every name and every expression in a program. Compilers use **type expressions** for this purpose. This section introduces the vocabulary that the rest of the note builds on.

1.1 What Is a Type Expression?

Definition. A **type expression** is either a basic type, or is formed by applying an operator called a **type constructor** to smaller type expressions. The set of basic types and constructors depends on the language being compiled.

The building blocks are:

- **Basic type:** a type expression on its own. Typical basic types are `boolean`, `char`, `integer`, `float`, and `void` (“the absence of a value”).
- **Type name:** a name (e.g. a class name) is itself a type expression.
- **Array constructor:** applying array to a number and a type expression builds a new type expression, e.g. `array(2, integer)`.
- **Record constructor:** a *record* is a data structure with named fields. Applying record to the field names and their types builds a record type; in practice this is implemented by applying record to a symbol table that holds the fields (Section 8).
- **Function constructor $\rightarrow$:** we write $s \rightarrow t$ for “function from type s to type t .”
- **Cartesian product $\times$:** if s and t are type expressions, $s \times t$ is a type expression (used to represent a list/tuple of types, e.g. function parameters). $\times$ associates to the left and binds tighter than $\rightarrow$.
- **Type variables:** a type expression may itself contain variables whose values are type expressions (used for compiler-generated type inference).

Worked Example — `int [2] [3]`

In C, the declared type `int [2] [3]` reads as “array of 2 arrays of 3 integers.” Its type expression is

```
array(2, array(3, integer))
```

The array operator takes two parameters — a number and a type — and, if types are represented as trees, returns a tree node labelled `array` with two children: the size and the element type.

Type expression tree for `int[2][3]`: `array(2, array(3, integer))`

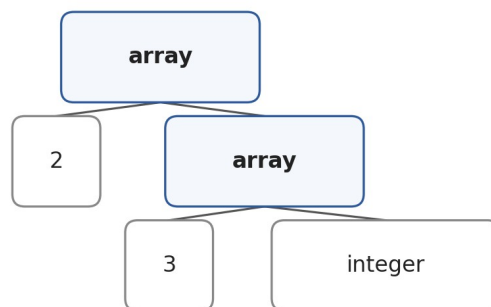


Figure 1.1 — Type-expression tree for `int[2][3]`: `array(2, array(3, integer))`.

1.2 Type Names and Recursive Types

Sidebar — Type Names and Recursive Types

Once a class is defined, its name can be used as a type name, e.g. `public class Node {...}` lets later code declare `public Node n;`.

Names can also define **recursive types**, needed for data structures such as linked lists. The pseudocode

```
class Cell { int info; Cell next; ... }
```

defines the recursive type `Cell` as a class containing a field `info` and a field `next` of type `Cell`. Similar recursive types can be built in C using records and pointers; the techniques in this note carry over to recursive types.

1.3 Representing Types as Trees or Graphs

A convenient implementation represents a type expression as a graph: interior nodes are type constructors and leaves are basic types, type names, or type variables (Figure 1.1 above is exactly such a tree). Because type names denote type expressions, they can set up implicit cycles — if an edge to a type name is redirected to the type expression it denotes, the resulting graph can have cycles, which is exactly how recursive types such as `Cell` are represented.

Q1. What is a type constructor? Give three examples.

A: A type constructor is an operator that builds a new type expression out of smaller type expressions (and possibly other parameters such as a number). Examples: `array (number × type → type)`, `record (field table → type)`, and `→ (type × type → type)`, for function types).

Q2. Why is `int[2][3]` written as `array(2, array(3, integer))` and not `array(3, array(2, integer))`?

A: C reads `int[2][3]` left to right as “array of 2 arrays, each of 3 integers,” so the outer array constructor takes the first bracket’s number (2) and an inner type that is itself `array(3, integer)`; reversing the order would describe `int[3][2]` instead.

Q3. How are recursive types such as a linked-list `Cell` represented as a type graph?

A: A type name (e.g. `Cell`) denotes a type expression. If the edge from a use of the name is redirected to the type expression it stands for, the graph develops a cycle back to the record node that declared the field of that type — this cycle is exactly what represents the self-reference in the recursive type.

2. Type Equivalence

Many type-checking rules have the form “if two type expressions are equal, return a certain type, else error.” Ambiguity arises once names are given to type expressions and later reused, so we need a precise notion of when two type expressions are the *same*.

2.1 Structural Equivalence

When type expressions are represented as graphs, two types are structurally equivalent under the following definition.

Definition — Structural Equivalence

Two type expressions are **structurally equivalent** if and only if one of the following holds:

- They are the same basic type.
- They are formed by applying the same constructor to structurally equivalent types.
- One is a type name that denotes the other.

2.2 Name Equivalence

If a type name is treated as standing for itself (rather than being expanded to what it denotes), the first two conditions above lead to a weaker notion, **name equivalence**. Name-equivalent expressions are assigned the same value number by the value-numbering method; structural equivalence, which is stricter to test because it must “see through” type names, can instead be tested with a unification algorithm.

Q1. State the three conditions for structural equivalence of type expressions.

A: Two type expressions are structurally equivalent if: (1) they are the same basic type, (2) they are built by applying the same type constructor to structurally equivalent operand types, or (3) one is a type name denoting the other.

Q2. What is the difference between name equivalence and structural equivalence?

A: Name equivalence treats a type name as standing only for itself, so two expressions are equivalent only if they use literally the same names/constructors (roughly conditions 1 and 2). Structural equivalence additionally lets a type name be replaced by the type expression it denotes (condition 3), so two differently-named but identically-structured types can still be judged equivalent.

3. A Grammar for Types and Declarations

To study types and declarations concretely, we fix a small grammar that declares one name at a time (lists of names, e.g. `int a, b, c;`, can be handled the same way, one name per declaration). This same grammar underlies **this entire note**: Sections 1–4 annotate it with true inherited/synthesized attributes, and Sections 5–9 show how those same attributes are computed in a single left-to-right pass suitable for bottom-up parsing.

Production	Meaning
$D \rightarrow T \text{ id } ; D \mid \epsilon$	D generates a sequence of declarations.
$T \rightarrow B C \mid \text{record } \{ ' D \}$	T generates a basic type, an array type, or a record type.
$B \rightarrow \text{int} \mid \text{float}$	B generates one of the basic types.
$C \rightarrow \epsilon \mid [\text{num}] C$	C (“component”) generates zero or more bracketed integers – the array dimensions.

In words:

- D generates a sequence of declarations, each of the form `T id ;`.
- T generates a basic type when it derives `B C` and C derives `ε`; otherwise C contributes one or more array dimensions, giving T an array type.
- B generates a basic type, `int` or `float`.

- C generates a string of zero or more integers, each surrounded by brackets, e.g. [2] [3].
- A record type (second production for T) is itself a sequence of field declarations D, surrounded by curly braces.

Q1. In the grammar $T \rightarrow B C \mid \text{record } \{ ' D ' \}$, what determines whether T is a basic type, an array type, or a record type?

A: If T derives B C with C deriving ϵ , T is a basic type. If C derives one or more [num] components, T is an array type built on B's basic type. If T instead derives record '{ ' D '}', T is a record type whose fields are the declarations generated by D.

Q2. Why can a single grammar (D, T, B, C) describe both plain variables and record fields?

A: A record's body is itself a D (a sequence of T id ; declarations), so the same declaration machinery that processes top-level variables is reused, recursively, to process the fields inside a record — only the symbol table and offset counter are switched out for the duration of that nested D.

4. Attributes for the Type Grammar

4.1 Why We Need an Inherited Attribute

Inherited attributes are useful precisely when the **shape of the parse tree** differs from the **shape of the abstract type** it must build. That mismatch is not caused by the parsing method — it comes from how the language is designed.

Consider `int [2] [3]`. Its type expression `array(2, array(3, integer))` is *deeply nested*: the outer array node contains the inner array node, which contains `integer`. But the parse tree for `int [2] [3]`, using $C \rightarrow [\text{num}] C$, is a *flat chain*: a C node for [2] next to a C node for [3], one hanging off the other. The basic type `integer`, produced by B at the *left* end of the tree, has to reach the *last* C node (the one that matches $C \rightarrow \epsilon$) before array-building can even start — and only then can the array type be assembled back up the chain.

This is exactly what an **inherited attribute** is for: it carries information *down and across* the tree (here, the basic type found at B, passed rightward through the chain of C nodes), while a **synthesized attribute** carries the assembled result back *up*.

4.2 The Attributes: B.t, T.t, C.b, C.t

Nonterminal	Attribute	Kind	Meaning
B	t	synthesized	the basic type (integer or float)
T	t	synthesized	the final type of the declaration
C	b	inherited	the basic type passed down the chain of C's
C	t	synthesized	the (possibly array) type accumulated so far

The inherited b attributes carry the basic type *down* the chain of C's; the synthesized t attributes accumulate the array type back *up* the chain.

4.3 Semantic Rules (SDD)

Production	Semantic Rules
$T \rightarrow B C$	$T.t = C.t$ $C.b = B.t$
$B \rightarrow \text{int}$	$B.t = \text{integer}$
$B \rightarrow \text{float}$	$B.t = \text{float}$
$C \rightarrow [\text{ num }] C_1$	$C.t = \text{array}(\text{num.val}, C_1.t)$ $C_1.b = C.b$
$C \rightarrow \epsilon$	$C.t = C.b$

Two remarks on the $C \rightarrow \epsilon$ rule (worth stating carefully, since it is easy to mis-copy): since this production has no C_1 child, its synthesized attribute simply **copies the inherited attribute unchanged**: $C.t = C.b$. This is the rule that finally “releases” the basic type that has been travelling down the chain, letting the array-building start on the way back up.

4.4 Worked Example — Annotated Parse Tree for int [2] [3]

We trace the SDD of §4.3 over the parse tree for the input string `int [2] [3]`.

- **Top-down pass (inherited b):** at the root $T \rightarrow B C$, B derives `int`, so $B.t = \text{integer}$. This value is copied into the outermost C as $C.b = \text{integer}$, and then passed unchanged, node after node, down the chain: each $C_1.b = C.b$ rule simply relays the same value `integer` to the next C .
- **Bottom-up pass (synthesized t):** at the rightmost, innermost C (the one matching $C \rightarrow \epsilon$), $C.t = C.b = \text{integer}$. Moving back up: the C for `[3]` computes $C.t = \text{array}(3, \text{integer})$; the C for `[2]` computes $C.t = \text{array}(2, \text{array}(3, \text{integer}))$; finally $T.t = C.t = \text{array}(2, \text{array}(3, \text{integer}))$ at the root.

Annotated parse tree (inherited attribute C.b, synthesized C.t) for int [2] [3]

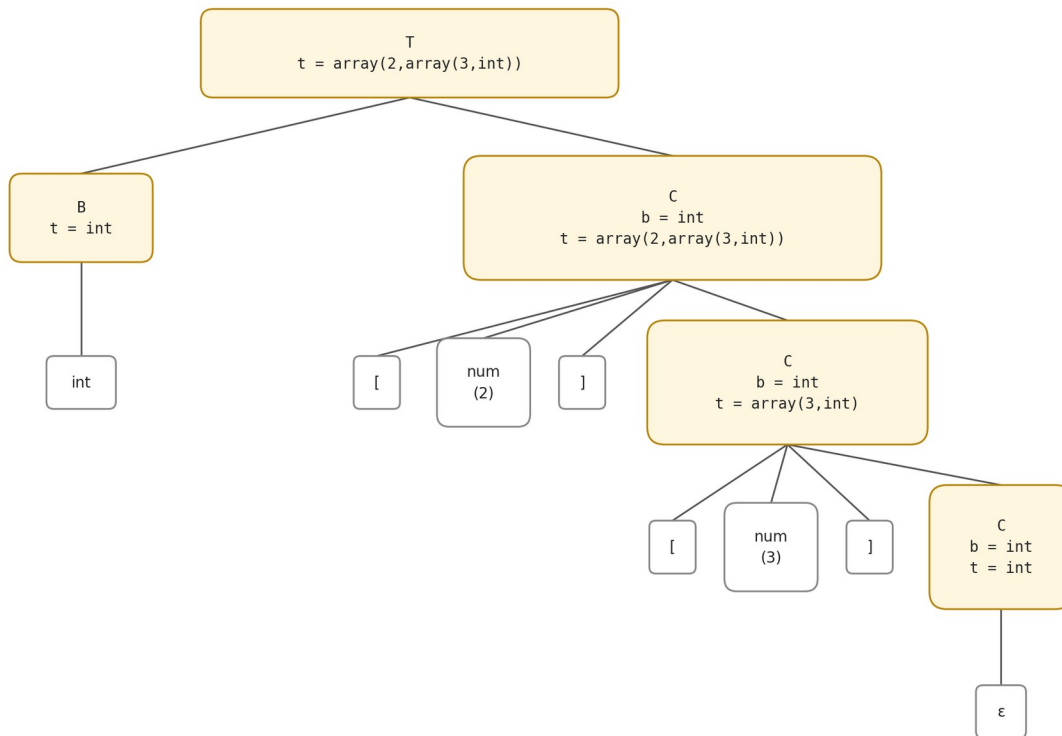


Figure 4.1 — Annotated parse tree for int [2] [3]: C.b flows down (inherited), C.t is assembled on the way back up (synthesized).

The final type expression, `array (2, array (3, integer))`, matches the tree of Figure 1.1 exactly — the SDD has translated a flat parse-tree chain into the correctly nested type-expression tree.

Q1. Why does the SDD give C both an inherited attribute b and a synthesized attribute t, instead of just one attribute?

A: b carries the basic type down the chain of C nodes so that every level knows what integer/float it is building an array of; t carries the assembled array type back up. A single attribute could not do both jobs at once because the value needed on the way down (a bare basic type) is not the value produced on the way up (a possibly deeply nested array type).

Q2. In the rule $C \rightarrow \epsilon \{ C.t = C.b \}$, why is this rule the turning point of the whole computation?

A: It is the only place where C.t is defined directly from C.b with no recursive C_1 involved, so it is where the inherited value that has been passed all the way down the chain is first converted into a synthesized value, letting the array(...) results build going back up.

Q3. Why is this SDD not S-attributed (synthesized-attributes-only)?

A: Because C.b is an inherited attribute: its value is copied down from a parent/left-sibling (B.t, then C.b of the enclosing C) rather than computed purely from the attributes of a production's own children, which is what an S-attributed definition requires.

5. From Inherited Attributes to Ordinary Variables

5.1 Why Bottom-Up Parsers Cannot Directly Use an Inherited Attribute

A bottom-up (shift-reduce / LR-style) parser executes a production's semantic action only at the moment that production is **reduced** — that is, only after *all* of the symbols on its right-hand side have already been parsed. This evaluation order is a perfect match for **synthesized** attributes, which are computed purely from the attributes of a production's children. It is a poor match for an **inherited** attribute such as $C.b$, whose value must already be available *before* the subtree that needs it is even reduced.

Fortunately, the particular inherited attribute in §4 has a very simple structure: $C.b$ is never computed from anything — it is simply **copied, unchanged**, from $B.t$ at the top, all the way down the chain of C nodes ($C_1.b = C.b$ at every step). A value that is only ever copied, never combined with anything, does not really need to be threaded through the grammar as a formal attribute at all — it can instead be held in an ordinary **variable** that is set once, immediately after B is reduced, and simply *read* by every later action, since B is always reduced (parsed) before the C chain that follows it in the input.

5.2 The Fix: Translation Scheme with Variables t and w

Figure 6.15's translation scheme (SDT) computes types **and** their widths for basic and array types using two ordinary variables, t and w , in place of what would be the inherited attributes of C . An embedded action between B and C in the production $T \rightarrow B C$ sets $t = B.type$ and $w = B.width$ the moment B has been reduced — before the parser ever starts reducing the C chain that follows. T and C keep ordinary **synthesized** attributes `type` and `width`, which *are* available to a bottom-up parser in the usual way.

Note — Translation Scheme vs. SDD

A **translation scheme** (SDT) is an SDD in which the actions are explicitly positioned within the right-hand sides of productions, rather than just attached to a production as a set of rules. Embedding an action between B and C like this can formally be modelled by introducing a **marker nonterminal** $M \rightarrow \epsilon$ at that position, so that $T \rightarrow B C$ becomes $T \rightarrow B M C$ with the action attached to $M \rightarrow \epsilon$. This is the same device used later, in §7.3, to place the `offset = 0` initialisation at the front of a declaration sequence.

5.3 Translation Scheme (Types and Widths)

Production	Semantic Action
$T \rightarrow B C$	<code>{ t = B.type; w = B.width; }</code>
$B \rightarrow \text{int}$	<code>{ B.type = integer; B.width = 4; }</code>
$B \rightarrow \text{float}$	<code>{ B.type = float; B.width = 8; }</code>
$C \rightarrow \epsilon$	<code>{ C.type = t; C.width = w; }</code>
$C \rightarrow [\text{num}] C_1$	<code>{ array(num.value, C₁.type); C.width = num.value × C₁.width; }</code>

The action between B and C runs the instant B is reduced, sets t and w , and those two values then sit waiting, unread, until the parser finally reduces the $C \rightarrow \epsilon$ production at the innermost end of the bracket chain — exactly mirroring the inherited $C.b$ of §4, but using an ordinary variable instead of a formal attribute.

5.4 Worked Example — Bottom-Up Evaluation for `int [2] [3]`

Reading `int [2] [3]` left to right, a bottom-up parser reduces in this order: $\text{int} \rightarrow B$, then the action sets $t = \text{integer}$, $w = 4$, then it shifts `[ 2 ]`, then `[ 3 ]`, then reduces the trailing ϵ to C (innermost), working back outward:

- C (innermost, ϵ): $\text{type} = t = \text{integer}$, $\text{width} = w = 4$.
- C for `[3]`: $\text{type} = \text{array}(3, \text{integer})$, $\text{width} = 3 \times 4 = 12$.
- C for `[2]`: $\text{type} = \text{array}(2, \text{array}(3, \text{integer}))$, $\text{width} = 2 \times 12 = 24$.
- $T \rightarrow B \ C$: $T.\text{type} = \text{array}(2, \text{array}(3, \text{integer}))$, $T.\text{width} = 24$.

Bottom-up evaluation (variables t, w instead of inherited attribute) for `int [2] [3]`

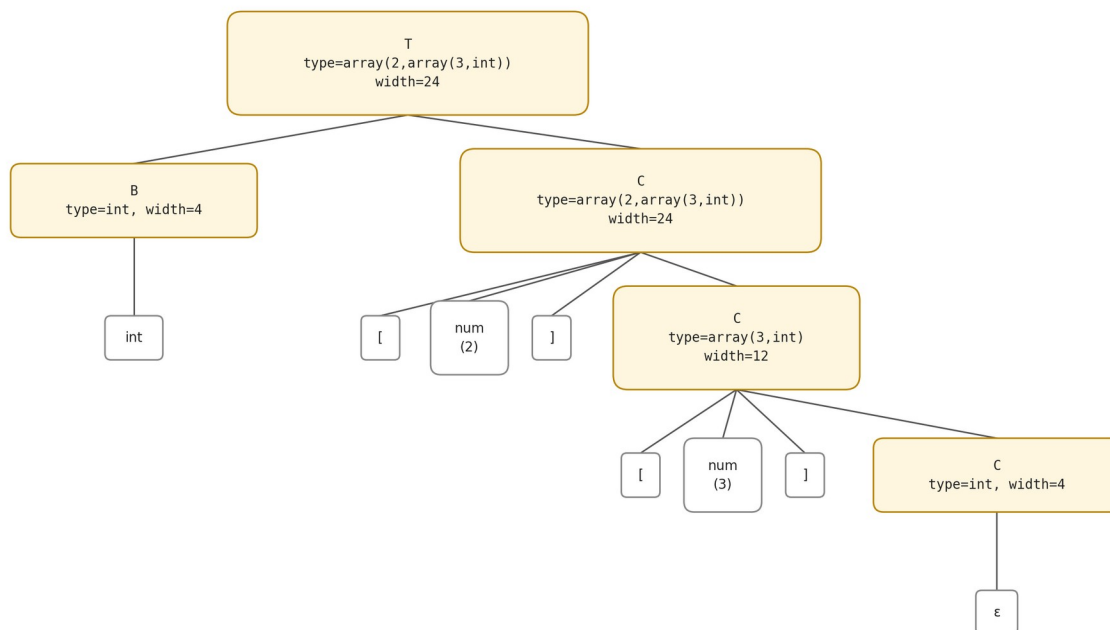


Figure 5.1 — Bottom-up evaluation of `int [2] [3]` using ordinary variables t, w in place of the inherited attribute $C.b$.

The type matches Section 4 exactly (Figure 4.1); the extra payoff here is the **width**: 24 bytes, i.e. $2 \times 3 \times 4$ (two arrays of three 4-byte integers each) — the subject of Section 6.

Q1. Why can the inherited attribute $C.b$ be replaced by an ordinary variable t in the bottom-up scheme, when in general inherited attributes cannot be evaluated bottom-up?

A: Because in this grammar $C.b$ is never computed from other values — it is only ever copied unchanged, all the way from $B.t$, down every level of the C chain. Since B is always parsed (and reduced) before any of the C 's that need its value, an ordinary variable set once right after B is reduced is read later by every action that needs it, with no need to pass it down as a formal attribute.

Q2. In Figure 6.15's SDT, why is the action $\{ t = B.\text{type}; w = B.\text{width}; \}$ placed between B and C in the production, rather than at the very end?

A: Because t and w must be assigned before the C chain starts being reduced, since $C \rightarrow \epsilon$ reads them. Placing the action at the end of the whole production (after C) would be too late — by then C 's actions, which need t and w , would already have executed with undefined values.

Q3. What are the type and width computed for the declaration `float [4][2]` using the SDT of §5.3?

A: $B \rightarrow \text{float}$ gives type = float, width = 8, so $t = \text{float}$, $w = 8$. The innermost C (ϵ) gets type = float, width = 8. The C for $[2]$ gives type = array(2, float), width = $2 \times 8 = 16$. The outer C for $[4]$ gives type = array(4, array(2, float)), width = $4 \times 16 = 64$. So $T.\text{type} = \text{array}(4, \text{array}(2, \text{float}))$ and $T.\text{width} = 64$.

6. Width of a Type and Storage Layout

6.1 Bytes, Words, and Width

Storage is organised as contiguous blocks of bytes, where a **byte** is the smallest unit of addressable memory (typically 8 bits); some fixed number of bytes form a **machine word**. A multibyte object is stored in consecutive bytes and identified by the address of its first byte.

The **width** of a type is the number of storage units (bytes) needed for an object of that type. A basic type — a character, integer, or float — needs an integral number of bytes; for easy access, an aggregate such as an array or a class is stored in one contiguous block of bytes.

6.2 Address Alignment and Padding

Sidebar — Address Alignment

Storage layout is strongly influenced by the addressing constraints of the target machine. Instructions that add integers, for instance, may expect integers to be **aligned** — placed at an address divisible by, say, 4. An array of ten `char`s needs only ten bytes, but a compiler may allocate 12 (the next multiple of 4), leaving two bytes unused.

Unused space left over for alignment is called **padding**. When space is at a premium, a compiler may instead **pack** data so that no padding is left; extra instructions may then be needed at run time to position the packed data so it can be treated as if properly aligned.

The alignment example later in Section 8 (Fall 2025, Question 8b) shows exactly this trade-off in a symbol-table computation: field offsets are rounded up to the next multiple of the field's own width before the field is inserted.

6.3 Why Array Width Is a Recursive Product of Dimensions

The rule $C.\text{width} = \text{num.value} \times C_1.\text{width}$ computes the width of an array *recursively*: the width of an array of n elements is n times the width of *one* element, and each element's own width is computed the same way one dimension further in.

This mirrors how a multi-dimensional array is actually laid out in memory: all the elements are stored contiguously, so the total number of storage units is simply the **product of every dimension's size**, times the width of the base (innermost) element —

$$\text{width}(T[d_1][d_2]\dots[d_k]) = d_1 \times d_2 \times \dots \times d_k \times \text{width}(T)$$

Building this product *recursively*, one bracket at a time from the innermost outward, is also exactly what is needed later (Chapter 6.4 of the textbook, outside this note's scope) to compute the *address* of an individual array element: if consecutive integers differ in address by 4, an index calculation for an array of integers must

include a multiplication by 4, and nested arrays require the product of all the *more deeply nested* dimensions as well. Making these multiplications explicit during width computation is what lets the front end later generate that addressing code, and it also gives an optimiser explicit multiplication operations it can strength-reduce.

Q1. What is padding, and why might a compiler introduce it?

A: Padding is storage space left deliberately unused so that a data object begins at an address that satisfies the target machine's alignment constraints (e.g. a multiple of 4). A compiler adds it because mis-aligned access to multi-byte data can be slower, or even illegal, on some architectures.

Q2. Why does the SDD compute array width by recursive multiplication ($C.width = num.value \times C1.width$) instead of, say, addition?

A: Because array elements are stored contiguously: an array of n elements occupies exactly n copies of one element's storage, so the total width is a product, not a sum. Applying this multiplicatively at every bracket level builds up the product of all dimension sizes times the base element's width, which is exactly the number of contiguous bytes the multi-dimensional array occupies.

Q3. If int is 4 bytes wide, what is the width of $int[3][4][2]$?

A: Multiply every dimension by the element width: $3 \times 4 \times 2 \times 4 = 96$ bytes.

7. Sequences of Declarations

Languages such as C and Java process all the declarations in a procedure as a group. We use a variable, `offset`, to track the next available relative address; each new name is entered into the symbol table at the current `offset`, which is then advanced by that name's width.

7.1 The offset Variable

Before the first declaration is considered, `offset` is set to 0. As each new name x is seen, x is entered into the symbol table with its relative address set to the current value of `offset`, which is then incremented by the width of x 's type.

7.2 Translation Scheme

Production	Semantic Action
$P \rightarrow D$	{ <code>offset = 0;</code> }
$D \rightarrow T \text{ id } ;$ D_1	{ <code>top.put(id.lexeme, T.type, offset);</code> <code>offset = offset + T.width;</code> }
$D \rightarrow \epsilon$	(no action)

The action inside $D \rightarrow T \text{ id } ; D_1$ creates a symbol-table entry by calling `top.put(id.lexeme, T.type, offset)`, where `top` denotes the current symbol table. `top.put` creates an entry for `id.lexeme` with type `T.type` and relative address `offset` in its data area.

7.3 Marker Nonterminals

The initialisation { `offset = 0;` } in the rule for P is really an action that must run *before* D is parsed — i.e. at the *start* of the production's right-hand side, not the end. Productions of the form $P \rightarrow \{ \text{action} \} D$ can be rewritten with a **marker nonterminal** M that derives only ϵ and carries the action:

```

P → M D
M → ε      { offset = 0; }

```

This is the same device already met in §5.2 for placing the `{ t = B.type; w = B.width; }` action between B and C. Because a marker nonterminal's production is reduced (its ϵ) the instant the parser reaches that point in the input, embedding an action this way lets a translation scheme run code at any position inside a production, not just at the very end — which is exactly what a bottom-up parser needs, since it can otherwise only run an action when the *whole* production is reduced.

Q1. Why is offset reset to 0 only once, at the very start of P, and not before every declaration?

A: Because each declaration's offset must continue on from where the previous one left off, so that every name gets a distinct relative address; offset is only reset to 0 when a brand-new, independent data area begins — which happens once for the whole procedure, and again (separately) at the start of each record's own field list, as shown in §8.

Q2. What two pieces of information does `top.put` store for each declared name, and where do they later get used?

A: It stores the name's type (`T.type`) and its relative address (offset). The type is needed for type checking and to compute widths of any type built on top of it; the relative address is needed to generate the correct memory-access code for the name at run time.

Practice Problem — Final Examination, Spring 2025, Set A | Question 1 [CO4] — Symbol-Table Attributes [10 marks]

Question: What are the attributes we store in the symbol table for each variable, and why do we need to store this information for code generation? Explain with examples.

Solution

For every declared name, the symbol-table entry built by the `D → T id ;` rule of §7.2 (extended with the record handling of §8.2) stores at least:

Attribute	What It Records	Why Code Generation Needs It
Name (<code>id.lexeme</code>)	the identifier's spelling, used as the table key	lets the compiler look the variable up every time it is referenced in an expression or statement
Type (<code>T.type</code>)	basic type, array type, or record(<code>env</code>)	determines how many bytes to reserve, which arithmetic/instruction variant to use (e.g. integer vs. float add), what implicit conversions are needed, and how to compute an array or field address
Width (<code>T.width</code>)	size in storage units computed from the type	needed both to lay out this variable's own space and to compute the offset of every name declared after it
Relative address / offset	the byte offset within the current data area (procedure activation record or record/class layout)	generated code addresses the variable as <code>base + offset</code> at run time, e.g. to load or store its value

Attribute	What It Records	Why Code Generation Needs It
Scope / symbol table pointer	which table (global, record, or nested record) the name lives in	the same name can exist in different scopes (Example in §7.3); the compiler must resolve each use to the correct declaration

Concretely, for `int a;` declared at offset 0, the entry (`a`, `integer`, width 4, offset 0) tells the code generator everything it needs to emit, e.g., `t1 = @base + 0 ; *t1 = 5` for an assignment `a = 5;`, without re-deriving the type or address from the declaration text at every use. For an array such as `int[3][5] z;` (Section 8), the stored width (60, from §6.3) is exactly what the compiler multiplies index expressions by when it later generates the address arithmetic for a reference such as `z[i][j]`.

8. Fields in Records and Classes

The translation of plain declarations (§7) carries over directly to fields inside records and classes, provided we are careful about two things.

8.1 Scoping of Field Names

- **Distinctness within a record:** a field name may appear at most once among the declarations generated by a single record's *D*.
- **Locally relative offsets:** the offset (relative address) of a field name is relative to the data area of *that record* — not to whatever data area was active outside it.

A consequence, illustrated in the worked example below: the same name used as a field of two different records (or as a plain variable outside any record) never conflicts, because each record keeps its own, completely separate symbol table.

8.2 Translation Scheme (Record Type)

A single extra production, added to the productions for *T* in §5–§7, handles record types. It has two semantic actions: one *before* *D* (opens a fresh scope) and one *after* *D* (closes it again).

Production	Semantic Action
<code>T → record '{'</code>	<code>{ Env.push(top); top = new Env(); Stack.push(offset); offset = 0; }</code>
<code>D '}'</code>	<code>{ T.type = record(top); T.width = offset; top = Env.pop(); offset = Stack.pop(); }</code>

Reading the pseudocode: `Env` implements symbol tables. `Env.push(top)` pushes the *currently active* symbol table onto a stack so it can be restored later; `top` is then pointed at a brand-new, empty table for the record's own fields. Symmetrically, `Stack.push(offset)` saves the enclosing scope's current offset, which is then reset to 0 for the new record. Once *D* (the field list) has been fully translated, `top` holds the types and relative addresses of every field, and `offset` holds the total storage the fields need; these become `T.type = record(top)` and `T.width = offset`, after which both `top` and `offset` are popped back to what they were *before* the record was entered — exactly a matched push/pop pair around the nested *D*, the same discipline used for entering and leaving any nested scope.

This same push/pop discipline is what makes it possible to process **nested** records: a record field that is itself a record simply triggers another, inner push/pop pair while the outer one is still suspended on the stack (see the Summer 2025 Q1.1 practice problem later in this section for a worked example two levels deep).

Worked Example — Exercise 6.3.1: Non-Conflicting Field Names

Determine the types and relative addresses for the identifiers in:

```
float x;
record { float x; float y; } p;
record { int tag; float x; float y; } q;
```

Applying §7.2 and §8.2 with standard widths (`int` = 4, `float` = 8):

Global table	Type	Width	Offset
x	float	8	0
p	record(Penv)	16	8
q	record(Qenv)	20	24
Penv (fields of p)	Type	Width	Offset
x	float	8	0
y	float	8	8
Qenv (fields of q)	Type	Width	Offset
tag	int	4	0
x	float	8	4
y	float	8	12

The three uses of `x` never conflict: global `x` sits at offset **0** in the global table; `p.x` sits at offset **0** within *Penv* (itself reached through `p` at global offset 8); and `q.x` sits at offset **4** within *Qenv* (reached through `q` at global offset 24). A subsequent statement `x = p.x + q.x`; therefore reads three genuinely different storage locations.

Q1. Why must `Env.push(top) / top = new Env()` happen before the record's field list `D` is translated, rather than after?

A: Because every `D → T id ;` rule inside the record must insert each field into the record's own, fresh table (not the enclosing scope's table) at an offset that starts from 0. If the push and the new table were created after `D`, the fields would be wrongly inserted into the enclosing scope and would wrongly continue the enclosing scope's offset.

Q2. What would go wrong if offset were not saved (pushed) before being reset to 0 for a record's fields?

A: Once the record's fields finished being processed, there would be no way to recover the enclosing scope's offset value, so the declaration that follows the record (e.g. the variable being declared with that record type) would be assigned the wrong relative address — typically it would continue from the record's own (much smaller) internal offset instead of from where the enclosing scope had actually reached.

Question: Using the SDT below, draw the parse tree, attribute calculations, and symbol-table contents for:

```
int y;
record {
  int[3][5] z;
  int y;
} q;
float z;
```

Given SDT (as printed on the paper):

Production	Semantic Rules
$D \rightarrow T \text{ id } ; D_1$	$\text{top.put(id.lexeme, T.type, offset);}$ $\text{offset} = \text{offset} + \text{T.width};$
$D \rightarrow \varepsilon$	(no action)
$T \rightarrow \text{record '{'}$ $D \text{ '}'}$	$\text{recEnv} = \text{top}; \text{recWidth} = \text{offset};$ $\text{top} = \text{envStack.pop()}; \text{offset} = \text{offsetStack.pop()};$ $\text{T.type} = \text{record}(\text{recEnv}); \text{T.width} = \text{recWidth};$
$B \rightarrow \text{int} \mid \text{float}$	$\text{B.type} = \text{integer (4)} \mid \text{float (8)}$
$C \rightarrow [\text{num}] C_1$	$\text{C.type} = \text{array}(\text{num.value}, \text{C}_1.\text{type}); \text{C.width} = \text{num.value}$ $\times \text{C}_1.\text{width}$
$C \rightarrow \varepsilon$	$\text{C.type} = \text{C.inType}; \text{C.width} = \text{C.inWidth}$

(The record production's opening action — $\text{envStack.push(top)}; \text{top} = \text{new Env()}; \text{offsetStack.push(offset)}; \text{offset} = 0;$ — and the standard widths $\text{int}=4$, $\text{float}=8$ are exactly as in §8.2.)

Part (a) — Solution

Processing the three top-level declarations left to right, with the global table G and $\text{offset} = 0$ initially:

- $\text{int } y;$: $\text{type} = \text{integer}$, $\text{width} = 4$. Insert y at offset 0. $\text{offset}: 0 \rightarrow 4$.
- $\text{record } \{ \text{int}[3][5] \text{ z}; \text{int } y; \} \text{ q};$: open a new scope ($\text{push G, 4}; \text{top} = \text{Q}, \text{offset} = 0$):
 - $\text{int}[3][5] \text{ z};$: $\text{type} = \text{array}(3, \text{array}(5, \text{integer}))$, $\text{width} = 3 \times (5 \times 4) = 60$. Insert z in Q at offset 0. $\text{offset}: 0 \rightarrow 60$.
 - $\text{int } y;$ (field): $\text{type} = \text{integer}$, $\text{width} = 4$. Insert y in Q at offset 60. $\text{offset}: 60 \rightarrow 64$.
 - Close the record scope: $\text{recEnv} = \text{Q}, \text{recWidth} = 64$; restore $\text{top} = \text{G}, \text{offset} = 4$. $\text{T.type} = \text{record}(\text{Q}), \text{T.width} = 64$.
- Back in G: insert q at offset 4 with width 64. $\text{offset}: 4 \rightarrow 68$.
- $\text{float } z;$: $\text{type} = \text{float}$, $\text{width} = 8$. Insert z in G at offset 68. $\text{offset}: 68 \rightarrow 76$.

Fall 2025 Q8(a): parse-tree summary for `int y; record{ int[3][5] z; int y; } q; float z;`

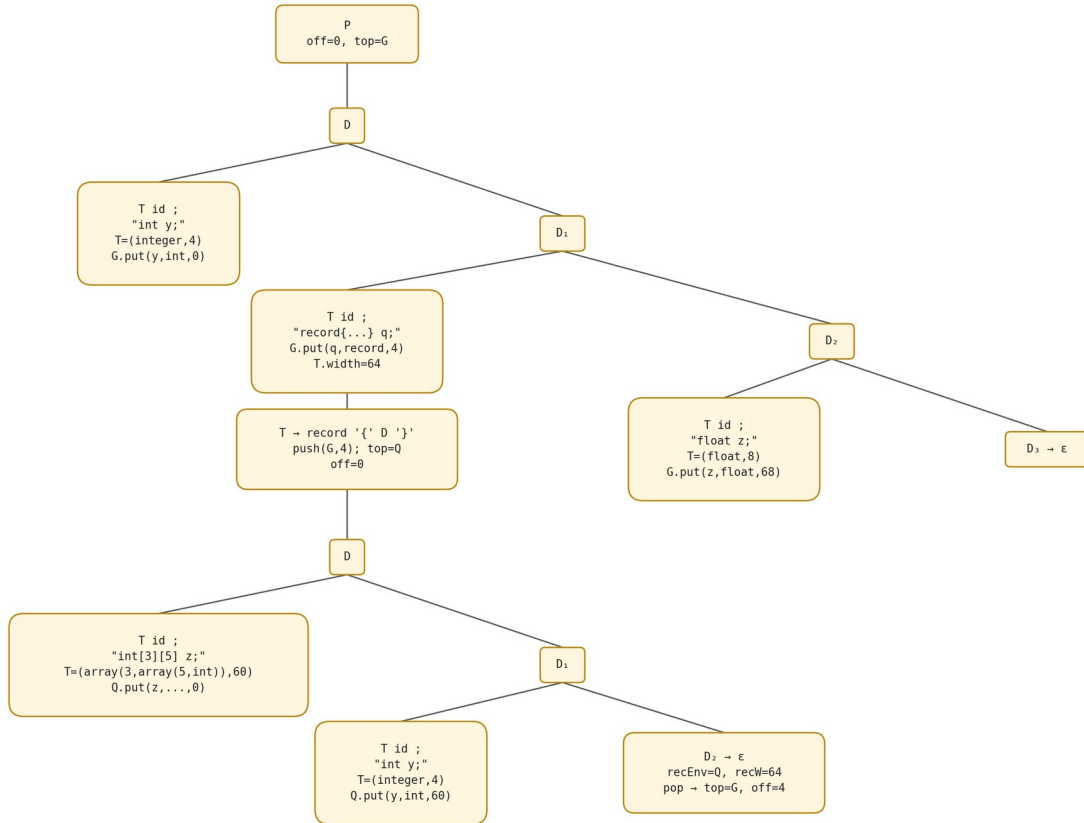


Figure 8.1 — Parse-tree summary for Fall 2025, Q8(a).

Stack Trace — `envStack` / `offsetStack`

The record's opening action `envStack.push(top); offsetStack.push(offset);` and its closing action `top = envStack.pop(); offset = offsetStack.pop();` form a single matched push/pop pair, since only one record (`q`) is entered here:

Stack trace — Fall 2025 Q8: entering/leaving record `q` (single push/pop)

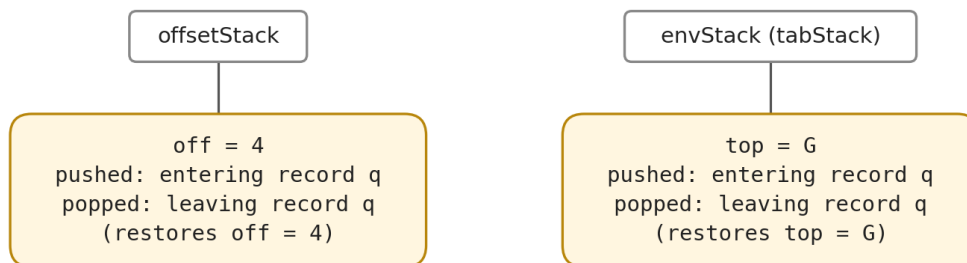


Figure 8.1s — Stack contents while inside record `q`'s field list.

Step	Action	offsetStack	envStack (tabStack)
1	before <code>D → T id ;</code> for <code>q</code> is reduced	[]	[]

Step	Action	offsetStack	envStack (tabStack)
2	push on entering record q	[4]	[G]
3	(process fields z, y inside Q)	[4]	[G]
4	pop on leaving record q	[] → offset = 4	[] → top = G

Global table G	Type	Width	Offset
y	integer	4	0
q	record(Q)	64	4
z	float	8	68

Q (fields of q)	Type	Width	Offset
z	array(3,array(5,integer))	60	0
y	integer	4	60

Part (b) — Solution (with alignment)

Now $D \rightarrow T \text{ id} ; D_1$ and the record-closing action are updated to round offset up, *before* inserting each name, using

```

align(n, k) = n,           if n % k == 0
              = n + (k - (n % k)), otherwise

D → T id ; { if (offset % T.width != 0) offset = align(offset, T.width);
              top.put(id.lexeme, T.type, offset);
              offset = offset + T.width; } D1
T → record '{' ... D '}' { ... T.width = align(recRawWidth, 8); ... }

```

Re-running the same declarations with these updated rules:

(The envStack/offsetStack push/pop mechanics are unchanged from the stack trace shown in part (a) above — alignment only changes *what value* gets inserted *after* the pop, not when the push/pop happens.)

- `int y;`: offset = 0, $0 \% 4 = 0$ — already aligned. Insert at 0. offset: $0 \rightarrow 4$.
- Record fields (z then y): inside the record offset starts at 0 again. z (width 60): $0 \% 60 = 0$, insert at 0, offset $\rightarrow 60$. y (width 4): $60 \% 4 = 0$, insert at 60, offset $\rightarrow 64$. **Both fields land at exactly the same offsets as part (a)**, since 60 and 64 already happen to be multiples of 4 — `recRawWidth = 64`.
- Record's own width is now `align(64, 8) = 64` (already a multiple of 8), so `T.width = 64` for q, unchanged.
- Back in G, about to insert q: current offset = 4, but `T.width = 64` and $4 \% 64 = 4 \neq 0$, so `offset = align(4, 64) = 4 + (64 - 4) = 64`. Insert q at offset **64** (not 4!). offset: $64 \rightarrow 64 + 64 = 128$.
- `float z;`: current offset = 128, width 8, $128 \% 8 = 0$ — already aligned. Insert z at offset $136 - 8 = 128$. offset: $128 \rightarrow 136$.

Only q and z's *global* offsets change (the record's internal field offsets are untouched, since they were already multiples of their own widths):

Global table G (aligned)	Type	Width	Offset
y	integer	4	0
q	record(Q)	64	64 (was 4)
z	float	8	136 (was 68)

(Table Q is unchanged from part (a): z@0, y@60, record width 64.) The gap between offset 4 and 64 left in front of q (60 unused bytes) is exactly the **padding** introduced so that q, an 8-byte-aligned 64-byte record, starts on a multiple of 8 — a direct application of the Address Alignment idea from §6.2.

Practice Problem — Final Exam, Summer 2025, Set A | Question 1.1 [CO4] [10 marks]

Question: Using the paper's full attribute grammar, draw the parse tree, annotate offset values at each D node, annotate type/width for each B, C, T node, and show the contents of every symbol table for:

```
int x;
record {
  int a;
  record {
    int b;
    float c;
  } w;
} y;
float[5] k;
int i;
```

The relevant fragment of the paper's SDT (standard widths, int=4, float=8, no alignment tricks this time):

Production	Semantic Rules
$D \rightarrow T \text{ id } ; D_1$	$\text{symbol.type} = T.\text{type}; \text{symbol.width} = T.\text{width};$ $\text{symbol.offset} = \text{off}; \text{top.insert}(\text{id.lexeme}, \text{symbol});$ $\text{off} = \text{off} + T.\text{width};$
$D \rightarrow \varepsilon$	(no action)
$T \rightarrow B \ C$	$T.\text{type} = C.\text{type}; T.\text{width} = C.\text{width}$
$B \rightarrow \text{int} \mid \text{float}$	$B.\text{type/width} = \text{int}, 4 \mid \text{float}, 8$
$C \rightarrow [\text{num}] \ C_1$	$C.\text{type} = \text{array}(\text{num.value}, C_1.\text{type}); C.\text{width} = C_1.\text{width} \times \text{num.value}$
$C \rightarrow \varepsilon$	$C.\text{type} = C.bT; C.\text{width} = C.bW$

Production	Semantic Rules
$T \rightarrow \text{record } \{$ $D \}$	<code>tabStack.push(top); offStack.push(off);</code> <code>top = new SymbolTable(); off = 0;</code> <code>... T.type = top; T.width = off;</code> <code>off = offStack.pop(); top = tabStack.pop();</code>

Solution

Processing the five top-level declarations left to right, global table G, off = 0 initially:

- `int x;` — type integer, width 4. `G.insert(x, 0).off: 0 → 4.`
- `record { int a; record { int b; float c; } w; } y;` — open scope (push G, 4; top = Y, off = 0):
 - `int a;` :: type integer, width 4. `Y.insert(a, 0).off: 0 → 4.`
 - `record { int b; float c; } w;` — open *another* scope (push Y, 4; top = W, off = 0):
 - `int b;` :: type integer, width 4. `W.insert(b, 0).off: 0 → 4.`
 - `float c;` :: type float, width 8. `W.insert(c, 4).off: 4 → 12.`
 - Close w's scope: `T.type = W, T.width = 12; restore top = Y, off = 4.` Back in Y: `Y.insert(w, type=W, width=12, offset=4).off: 4 → 16.`
- Close y's scope: `T.type = Y, T.width = 16; restore top = G, off = 4.` Back in G: `G.insert(y, type=Y, width=16, offset=4).off: 4 → 20.`
- `float[5] k;` — B.type=float, width=8; C for [5]: type=array(5, float), width = $8 \times 5 = 40$. `G.insert(k, width=40, offset=20).off: 20 → 60.`
- `int i;` :: type integer, width 4. `G.insert(i, offset=60).off: 60 → 64.`

Summer 2025 Q1.1: parse-tree summary for `int x; record{int a; record{int b; float c;} w;} y; float[5] k; int i;`

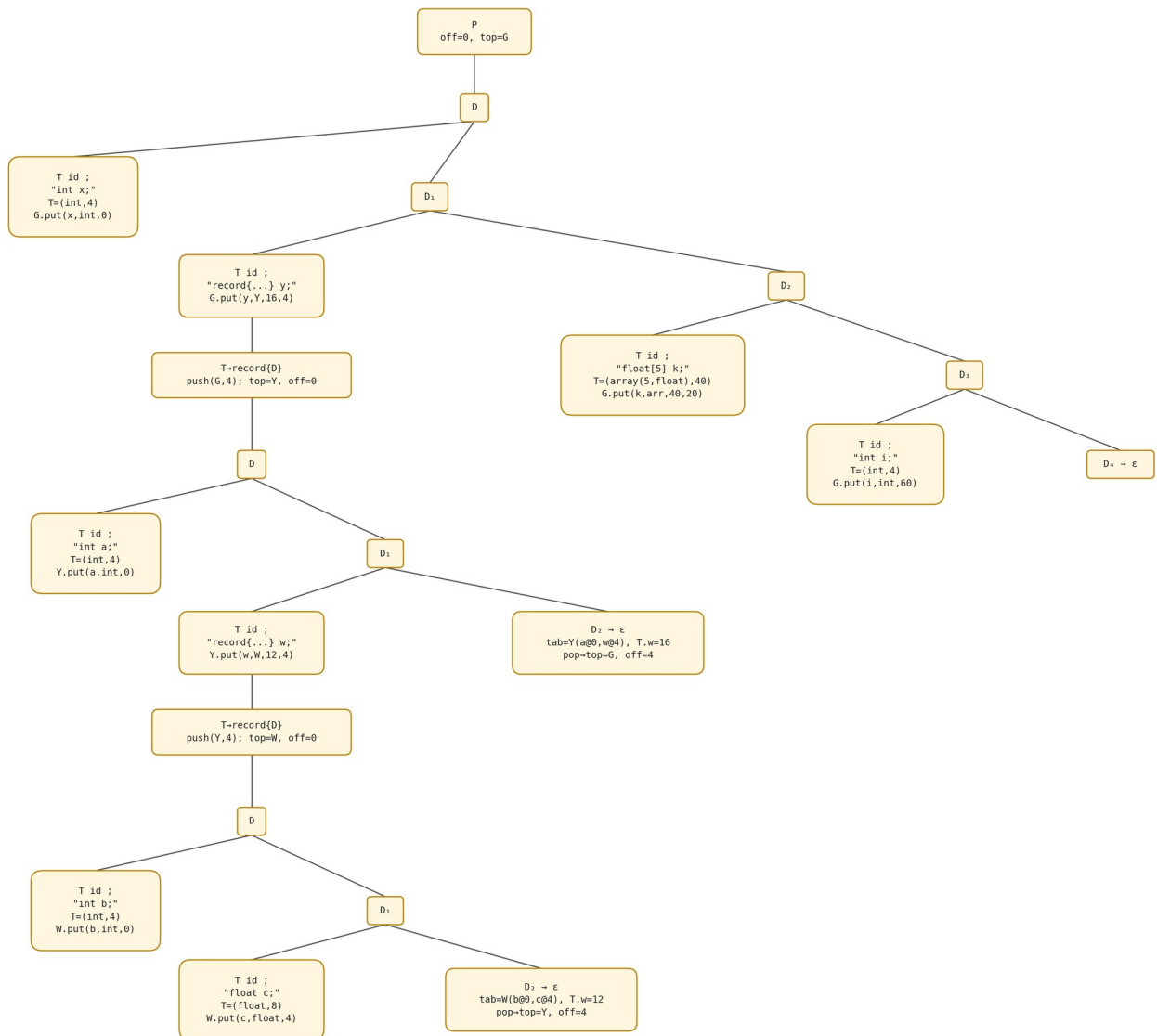


Figure 8.2 — Parse-tree summary for Summer 2025, Q1.1 (nested records y and w).

Stack Trace — tabStack / offsetStack (nested, depth 2)

Here the scope stack genuinely **nest**s: entering `w` happens *while* `y`'s own push is still sitting on the stack, unresolved. The two pops must therefore happen in **reverse** (last-in-first-out) order — `w`'s frame first, then `y`'s:

Stack trace — Summer 2025 Q1.1: record w nested inside record y (depth 2)

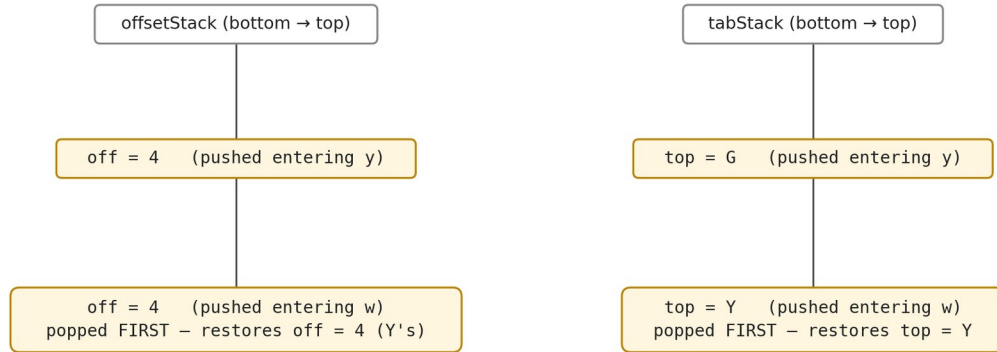


Figure 8.2s — Stack contents at maximum depth, while inside w's field list (nested inside y's).

Step	Action	offsetStack	tabStack
1	before entering y	[]	[]
2	push on entering record y	[4]	[G]
3	push on entering record w (nested)	[4, 4]	[G, Y]
4	pop on leaving w — first pop	[4] → off = 4	[G] → top = Y
5	pop on leaving y — second pop	[] → off = 4	[] → top = G

Global table G	Type	Width	Offset
x	integer	4	0
y	record(Y)	16	4
k	array(5,float)	40	20
i	integer	4	60

Y (fields of y)	Type	Width	Offset
a	integer	4	0
w	record(W)	12	4

W (fields of w, nested inside y)	Type	Width	Offset
b	integer	4	0
c	float	8	4

Notice how the scope stack grows two levels deep for w (inside y, itself a top-level declaration) and unwinds cleanly: each `tabStack.push / top = new SymbolTable()` pair is undone by exactly one matching pop before its enclosing declaration is finished — the same discipline as §8.2, simply nested one level deeper.

Q1. Why does the offset for k (20) not start from 0, even though k is a completely different type of declaration from the record y that precedes it?

A: Because off in the global table G is a single running counter across every top-level declaration, regardless of each declaration's own type; it only gets reset to 0 for the duration of a nested record's own field list (a separate table), never for a new declaration in the same scope.

Q2. If a fourth field, int d;, were added inside w (after c), what would its offset be, and what would w's new width become?

A: d would be inserted at the current off inside W, which is 12 (right after c's 8-byte float ending at offset 12), giving d width 4 at offset 12; w's own width (T.width = off) would become 16, and w's offset inside y and every offset after it would be unaffected, since y's own off only advances once, when w as a whole is inserted.

Practice Problem — Final Exam, Spring 2026, Set A | Question 7 [CO4] [5 + 2 = 7 marks]

Caution — This SDT Deliberately Uses Non-Standard Numbers

Unlike every other example in this note, this paper's grammar assigns `int` a width of **3** and `float` a width of **6**, and it adds an unusual `+5` and `×2` to the width formulas. These are *not* real machine widths — they exist purely to test whether the SDT's rules are applied exactly as written, rather than from memorised “standard” values. Solve mechanically, rule by rule.

Question (a): draw the parse tree, attribute calculations, and symbol table for:

```
int a;
record {
  int[5][10] x;
} d;
```

Given SDT:

Production	Semantic Rules
P	tab = new SymbolTable(); off = 500; stTab, stOff = new Stack()
$D \rightarrow T \text{ id } ; D_1$	tab.insert(id.lexeme, {type:T.t, width:T.w, offset:off}); off = off + T.w
$D \rightarrow \epsilon$	(no action)
$T \rightarrow B \ C$	C.bt = B.t; C.bw = B.w; T.t = C.t; T.w = C.w
$T \rightarrow \text{record '}'$ D '}'	stTab.push(tab); stOff.push(off); tab = new SymbolTable(); off = 0; T.t = tab; T.w = off × 2; tab = stTab.pop(); off = stOff.pop();
$B \rightarrow \text{int}$	B.t = i; B.w = 3

Production	Semantic Rules
$B \rightarrow \text{float}$	$B.t = f; B.w = 6$
$C \rightarrow [\text{num}] C_1$	$C.t = \text{array}(\text{num.value}, C_1.t); C.w = \text{num.value} \times C_1.w + 5$
$C \rightarrow \epsilon$	$C.t = C.bt; C.w = C.bw$

Part (a) — Solution

Note the initial offset here is $\text{off} = 500$, not 0 — taken exactly as the paper's P rule states.

- $\text{int } a; B \rightarrow \text{int}$ gives $t = i, w = 3$. $C \rightarrow \epsilon$ gives $C.t = i, C.w = 3$. So $T.t = i, T.w = 3$. Insert a at offset 500. $\text{off}: 500 \rightarrow 503$.
- $\text{record } \{ \text{int}[5][10] x; \} d; \text{push}(\text{tab}=G, \text{off}=503); \text{tab} = R(\text{new table}), \text{off} = 0$.
 - $\text{int}[5][10] x; B \rightarrow \text{int}: t = i, w = 3$. Working from the **innermost** bracket outward:
 - Innermost $C \rightarrow \epsilon$ (past $[10]$): $C.t = i, C.w = 3$.
 - C for $[10]$: $C.t = \text{array}(10, i), C.w = 10 \times 3 + 5 = 35$ (not 30 — remember the +5!).
 - C for $[5]$: $C.t = \text{array}(5, \text{array}(10, i)), C.w = 5 \times 35 + 5 = 180$.
 - So $T.t = \text{array}(5, \text{array}(10, i)), T.w = 180$. Insert x into R at offset 0. $\text{off}: 0 \rightarrow 180$.
 - Close the record: $T.t = \text{tab} = R, T.w = \text{off} \times 2 = 180 \times 2 = 360$ (the $\times 2$ rule — not simply 180!). Restore $\text{tab} = G, \text{off} = 503$.
- Back in G : insert d at offset 503 with type R and width 360. $\text{off}: 503 \rightarrow 863$.

Spring 2026 Q7(a): parse-tree summary for `int a; record{ int[5][10] x; } d;`

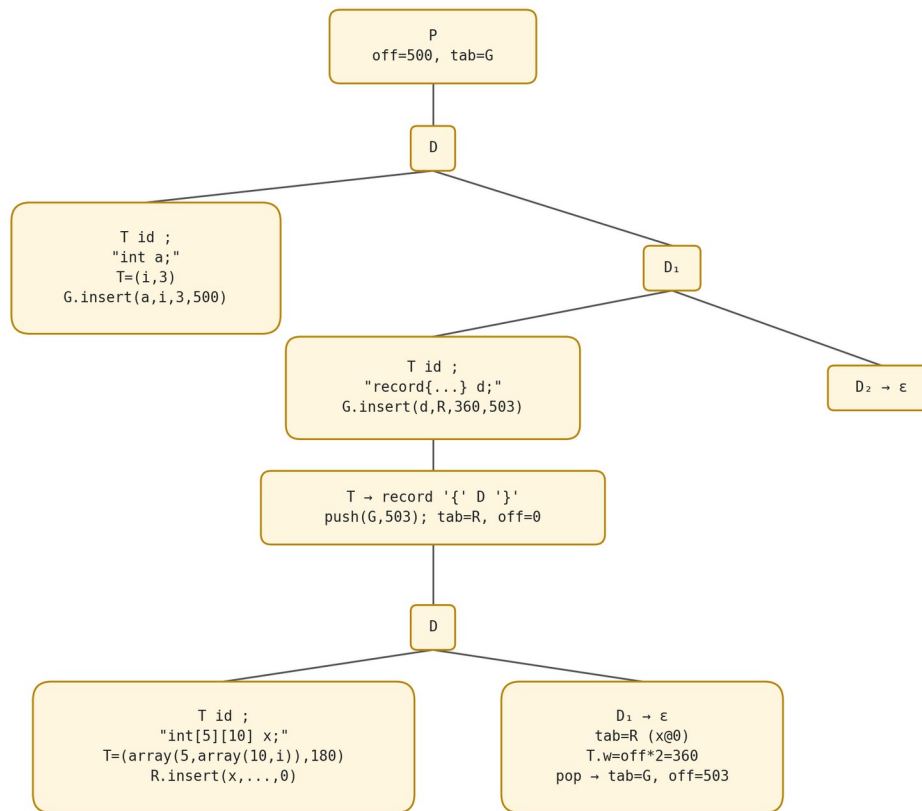


Figure 8.3 — Parse-tree summary for Spring 2026, Q7(a). Note the non-standard widths (`int = 3`) and the `+5 / ×2` adjustments.

Stack Trace — `stTab` / `stOff`

As with Fall 2025 Q8, only one record (`d`) is entered, so `stTab`/`stOff` see a single matched push/pop pair — note the pushed offset is `503`, not `0`, since `a` was already declared before `d`:

Stack trace — Spring 2026 Q7: entering/leaving record `d` (single push/pop)

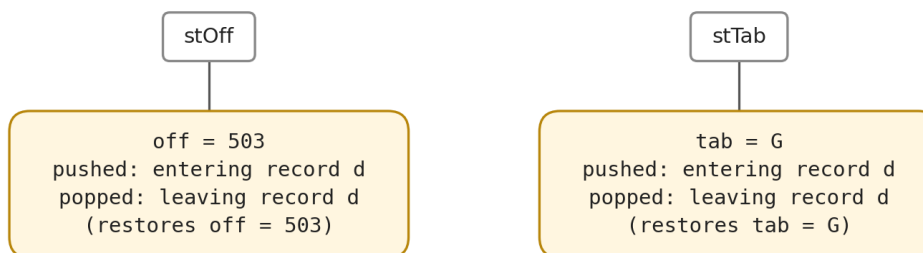


Figure 8.3s — Stack contents while inside record `d`'s field list.

Step	Action	stOff	stTab
1	before <code>D → T id ;</code> for <code>d</code> is reduced	[]	[]

Step	Action	stOff	stTab
2	push on entering record d	[503]	[G]
3	(process field x inside R)	[503]	[G]
4	pop on leaving record d	[] → off = 503	[] → tab = G

Global table G (tab)	Type	Width	Offset
a	i	3	500
d	record(R)	360	503

R (fields of d)	Type	Width	Offset
x	array(5,array(10,i))	180	0

Part (b) — Solution

Question: Explain why syntax-directed translation computes array width using recursive multiplication of dimensions and element width.

An array's elements are stored contiguously, so its total storage is the **number of elements** times the storage each element needs. For a multi-dimensional array, the “number of elements” is itself the product of every dimension's size, and the width of one “element” at each level is the (already-computed) width of everything nested *inside* that dimension. Computing `C.width` recursively — one bracket at a time, multiplying the current dimension's `num.value` by the width already synthesized for the remaining, more deeply nested dimensions (`C1.width`) — builds up exactly this product, level by level, using only the information locally available at each `C` node. (This particular paper additionally folds in a constant +5 per bracket, but the *multiplicative* structure `num.value × C1.w` is what does the real width-accumulation work; see §6.3 for the standard version of this rule.)

Q1. In this SDT, why does inserting `d` use `off = 503` rather than `off = 0`?

A: `d` is a top-level (global) declaration, so it is inserted using the global table's own off, which had already advanced to 503 after `a` was declared. The `off = 0` reset only applies inside the record's own nested `D`, for the record's internal fields (here, `x`).

Q2. What would `T.w` for `d` have been if the record rule used `T.w = off (no ×2)`, all else unchanged?

A: It would be 180 instead of 360 — exactly half — since off after processing the record's fields is 180 regardless of what the closing rule does with it; the `×2` in this paper's rule is applied only at the very end, once, to the raw field-total off.

Practice Problem — Final Exam, Spring 2026, Set A | Question 4 [CO5] [1.5 + 1.5 + 2 = 5 marks]

Question: given the four struct definitions below and a declared variable `S studentRecord;`, with `int = 4` bytes and `char = 1` byte, and `@studentRecord` denoting its base address:

```
struct P { char grade; int marks; char section[6]; };
struct Q { int courseId; P result; char room[5]; };
struct R { char title[8]; Q course; int semester; };
struct S { int studentId; R info; char university[12]; };
```

- (a) Determine the total offset of the field accessed in `y = studentRecord.info.course.result.section[4]`; relative to `@studentRecord`.
- (b) Write the equivalent three-address code for the statement above.
- (c) Describe how subclasses in inheritance are represented in memory layout during compilation, and explain with an example why a stack might be necessary to implement this.

Part (a) — Solution

Exactly as in §7–§8, each struct's fields are laid out contiguously, one after another, so each field's offset is the sum of the widths of every field before it *within that struct*. Working from the innermost struct P outward:

struct P	Type	Width	Offset in P
grade	char	1	0
marks	int	4	1
section[6]	char[6]	6	5

Total width of P = $1 + 4 + 6 = 11$ bytes.

struct Q	Type	Width	Offset in Q
courseld	int	4	0
result	P	11	4
room[5]	char[5]	5	15

Total width of Q = $4 + 11 + 5 = 20$ bytes.

struct R	Type	Width	Offset in R
title[8]	char[8]	8	0
course	Q	20	8
semester	int	4	28

Total width of R = $8 + 20 + 4 = 32$ bytes.

struct S	Type	Width	Offset in S
studentId	int	4	0
info	R	32	4
university[12]	char[12]	12	36

Total width of S = $4 + 32 + 12 = 48$ bytes.

The access `studentRecord.info.course.result.section[4]` chains through four field-offsets and one array index, each relative to the *start of its own immediately-enclosing struct* — exactly the scoping rule of §8.1, applied to structs instead of records:

Step	Field / Index	Local Offset	Running Total
1	S.info	4 (within S)	4
2	R.course	8 (within R)	4 + 8 = 12
3	Q.result	4 (within Q)	12 + 4 = 16
4	P.section	5 (within P)	16 + 5 = 21
5	section[4]	4 × 1 (char width)	21 + 4 = 25

The total offset of `section[4]` relative to `@studentRecord` is **25 bytes**.

Part (b) — Solution

Every field offset and the array index used here are known at compile time (none depends on a run-time value), so the whole chain of additions can be generated in either of two equivalent forms.

Unoptimised (mirroring the field-access chain one step at a time, in the style of the L nonterminal's `gen(...)` rules used elsewhere in this course):

```

t1 = @studentRecord + 4    // + offset of info in S
t2 = t1 + 8                // + offset of course in R
t3 = t2 + 4                // + offset of result in Q
t4 = t3 + 5                // + offset of section in P
t5 = 4 * 1                 // section[4]: index * element width
t6 = t4 + t5
y  = *t6

```

Optimised (constant folding — since every one of 4, 8, 4, 5 and 4×1 is a compile-time constant, the compiler can add them once, at compile time, into a single displacement):

```

t1 = @studentRecord + 25
y  = *t1

```

Part (c) — Solution

A subclass's object is laid out exactly like a record whose declarations are the *union*, in order, of its superclass's fields followed by its own newly-added fields: the superclass's fields keep **exactly the relative addresses they were assigned while laying out the superclass** (so that code written against the superclass, expecting field `f` at offset `k`, still works unmodified on a subclass object), and the subclass's own fields are appended contiguously afterward, continuing the same `offset` counter (§7.1) from where the superclass layout left off. This is precisely Exercise 6.3.2's extension of §8.2's record-field handling to classes and single-inheritance hierarchies.

A stack is necessary for essentially the same reason it is necessary for nested records in §8.2: laying out a class or record requires **temporarily switching** to a fresh symbol table and a fresh `offset` counter for its own field list, while the enclosing scope's table and `offset` must be preserved unchanged and restored afterward. Whenever one class's layout is nested inside another's processing — a superclass being laid out before its subclass, or a class containing a field whose type is itself another class or record — these push/pop pairs **nest**, and only a *stack* (last-in-first-out) correctly restores each enclosing scope's table and `offset` in the right order once the inner one finishes, exactly as `envStack/offsetStack` (or `Env.push/Stack.push`) do in §8.2 and in the Summer 2025 Q1.1 example earlier in this section, which nests record inside record two scopes deep.

Concretely: suppose class `Base` has fields `int a`; (offset 0) and `float b`; (offset 4), total width 12, and class `Derived` extends `Base` adds `int c`; . Laying out `Derived` pushes the current (outer) scope, processes `Base`'s fields into a fresh table, then — rather than popping back to the outer scope — *continues* inserting `Derived`'s own field `c` into that *same* table at `offset = 12` (continuing `Base`'s own final offset, not resetting to 0), before finally popping back to the scope that was active before `Derived` itself was declared.

Q1. Why can the compiler fold the whole offset chain of Q4(a) into the single constant 25, but generally cannot do this for an array access such as `z[i][j]` (§6.3)?

A: Every offset and index used in Q4(a) is a literal constant known at compile time, so the additions can be performed once, during compilation. `z[i][j]` involves the run-time values of variables `i` and `j`, so the address arithmetic (multiplying each index by the appropriate dimension width and adding) cannot be evaluated until the program actually runs, and must be emitted as executable three-address code instead.

Q2. In the inheritance layout of Q4(c), why must a subclass's inherited fields keep the exact same offsets they had in the superclass, rather than being freely reassigned?

A: So that code compiled against the superclass's declared field layout (which addresses each field as `base + offset`) continues to work correctly when it is handed a subclass object at run time — a subclass object must be usable anywhere a superclass object is expected, which requires its inherited fields to sit at identical offsets.

9. Single-Pass Bottom-Up Processing — Practical Implications

Every translation scheme in this note (§5, §7, §8) shares one property: it computes types, widths, and symbol-table entries in a **single left-to-right, bottom-up pass** over the input, using only ordinary variables and stacks in place of true inherited attributes. This is what makes the schemes implementable directly inside an LR parser's reduce actions, with no second pass and no separate tree-walking phase. That efficiency is not free — it constrains what the source language is allowed to look like.

9.1 What a Single Post-Order Pass Requires

If symbol-table construction, scope analysis, *and* type checking of expressions must **all** happen together, in one post-order (bottom-up) traversal of the syntax tree, every fact a node's semantic action needs must already be available by the time that node is visited — i.e., must have been produced by some node visited *earlier* in the same traversal. This rules out, or at least significantly restricts, source-language constructs whose meaning depends on something that textually or structurally comes *later*:

- **Declare-before-use:** a name must be declared before it is referenced, since the single pass visits a use only after (or, for expressions inside the declaration's own subtree, never before) it has visited the corresponding declaration; forward references to a variable declared later in the same scope cannot be resolved without a second pass or a pre-pass that just collects declarations.
- **No mutual/forward recursion between types or classes:** if class `A` refers to class `B` and `B` also refers to `A`, a single pass processing declarations in file order cannot type-check whichever one is encountered first, because the type of the not-yet-seen class does not exist yet.
- **No use of a function/procedure before its signature is known:** a call's type check needs the callee's parameter and return types already recorded, again forcing declare-before-use for procedures, not only variables.

Languages that *do* allow forward references (e.g. Java allows a class's method to call another method defined later in the same class) support this by giving up the single-pass guarantee: a real compiler for such a language first makes a quick pass purely to collect top-level declarations into the symbol table (populating names and

headers only), and only *then* performs the full post-order pass that does scope resolution and type checking against that already-complete table.

Q1. Why does requiring symbol-table generation, scope analysis, and type checking in one post-order traversal force declare-before-use?

A: In a post-order (bottom-up) traversal, a node's semantic action runs strictly after all of its children (and everything visited before it in traversal order) have already been processed. If a name's use is checked against a symbol-table entry that a later declaration would create, that entry does not exist yet at the time the use is visited, so the check cannot succeed within a single pass; the language must therefore require every name to be declared earlier in the traversal than any place it is used.

Q2. How can a compiler support forward references (e.g. mutually recursive classes) while still keeping type checking itself a single bottom-up pass?

A: By separating declaration-collection from type checking: a first, lightweight pass walks the program only to register every top-level name and its signature/type header in the symbol table (without checking bodies), and a second pass then performs the full post-order symbol-table-lookup-and-type-check traversal, by which point every name it might need — including ones declared later in the source — is already present in the table.

Practice Problem — Final Examination, Fall 2025, Set A | Question 5 [CO2] [2 marks]

Question: If symbol table generation and scope analysis and type checking of expressions in a program must be done in a single post-order traversal of the abstract syntax tree, how does it restrict the structure of the source program, and why?

Solution

It restricts the source program to be **declare-before-use** everywhere — every name (variable, function, class/type) must be declared, and its declaration fully processed, before the traversal reaches any point that uses that name. This applies not only to ordinary variables but to types and procedures as well, which rules out, in particular, **mutually recursive type or class declarations** and **forward calls** to a not-yet-declared procedure.

The reason is the evaluation order of a post-order traversal itself: a node's action (inserting into the symbol table, resolving a name's scope, or checking an expression's type) runs only *after* every node visited earlier in the traversal — which, for a program read and parsed in its natural top-to-bottom, left-to-right order, means only after everything *textually before* it. A single pass has no way to “look ahead” to a declaration that has not been visited yet; the symbol-table entry a later declaration would create simply does not exist at the moment an earlier use needs it. This is exactly the same constraint already seen mechanically throughout §5–§8 of this note: every SDT here (types and widths in §5, offset in §7, Env/Stack push-pop for records in §8) is deliberately built so that each action only ever needs values already computed earlier in the same left-to-right pass — which is precisely what makes single-pass, bottom-up compilation possible for them in the first place.